

CS1CA Computer Systems Architecture

Processor Pipelining (3)

Week 9 lecture 1 of autumn semester 2025

Lectured by

Dr. Hong Wei

h.wei@reading.ac.uk

Room 133 in Polly Vacher

Instr No.	Pipeline Stage						
1	IF	ID	EX	MEM	WB		
2		IF	ID	EX	MEM	WB	
3			IF	ID	EX	MEM	WB
4				IF	ID	EX	MEM
5					IF	ID	EX
Clock Cycle	1	2	3	4	5	6	7

Outline of the topic

- Introduction to pipelining
- The basic pipeline for DLX machine
- Performance issues in pipelining
- Pipeline hazards
 - Data hazards
 - Structure hazards
 - Control hazards
- DLX pipeline to handle multi-cycle operations
- Summary: the MIPS R4000 pipeline (self-learning)

In this lecture

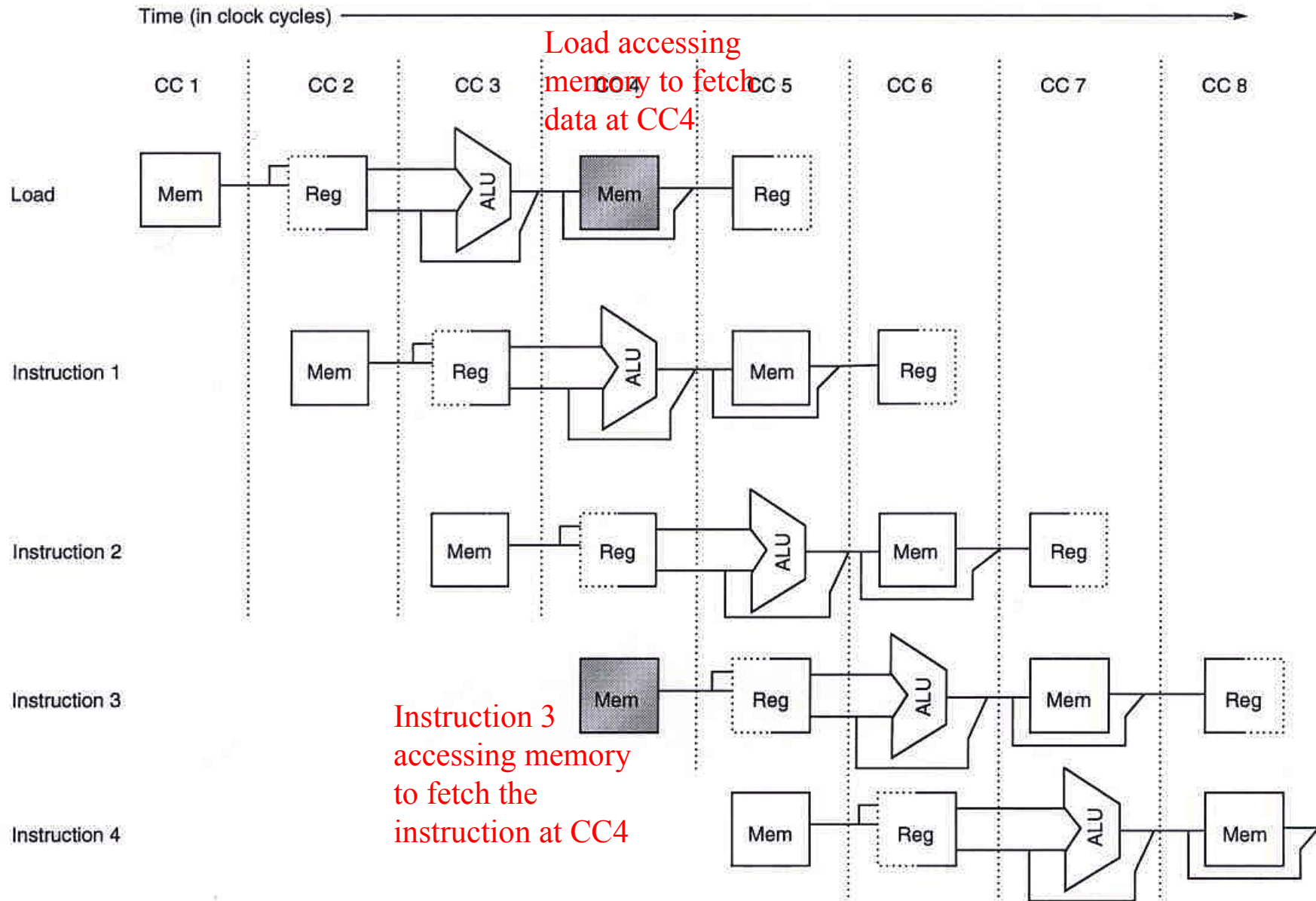
- The following contents will be delivered
 - Pipeline hazards
 - Structure hazards
 - Control hazards
- The learning outcomes are listed below
 - To be able to identify structure hazards and control hazards
 - To be able to use various techniques to deal with structure hazards and control hazards with pipeline timing tables

Structure Hazards

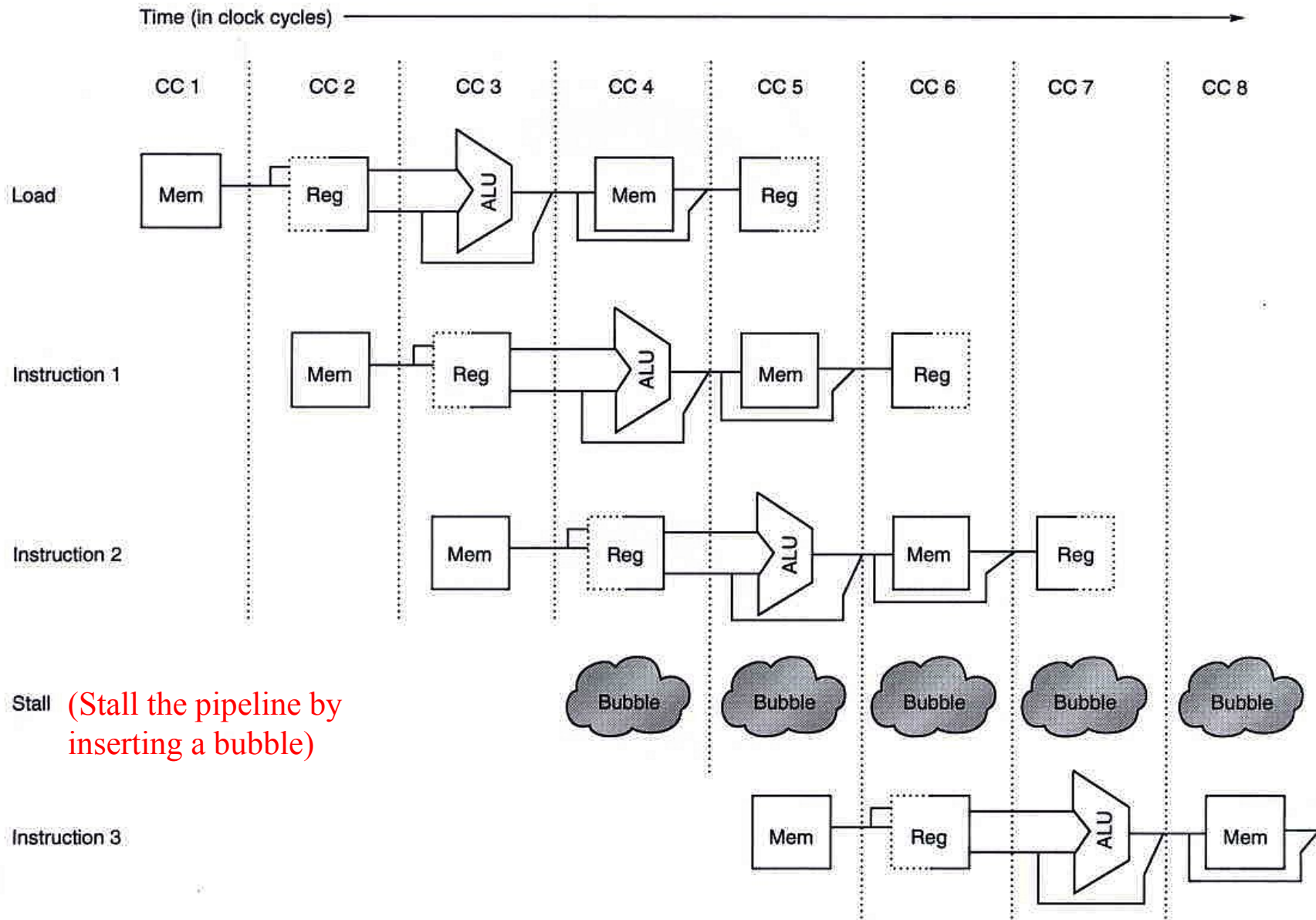
- In a pipelined machine, if combination of instructions cannot be accommodated in one clock cycle because of resource conflicts, the machine is said to have a structure hazard.
- To stall the pipeline for one clock cycle will solve the problem. This will increase CPI, as shown in the following example.

A stall is commonly called a pipeline bubble.

A machine with only one memory port will generate a conflict when IF and MEM perform in the same clock cycle.



The structure hazard causes inserting pipeline bubbles.



Example

- Data reference of 40% of the mix (40% load/store instructions);
- Ideal CPI of pipelined instruction is 1;
- Machine with structural hazard has a **clock rate** that is 1.05 times higher than the clock rate of the machine without the hazard.
- Which is faster?

Note: Load/store will introduce structure hazards when using one memory port.

Average instruction execution time (AIET) = CPI × clock cycle time

For the machine without stall: $AIET_{\text{no structural hazard}} = \text{clock cycle time}_{\text{ideal}}$

For the machine with structural hazard:

The ideal CPI=1

$$AIET_{\text{structural hazard}} = (1 + 0.4) \times \frac{\text{clock cycle time}_{\text{ideal}}}{1.05}$$
$$= 1.3 \times \text{clock cycle time}_{\text{ideal}}$$

CPI with stall cycles

Machine without
structure hazard is
faster.

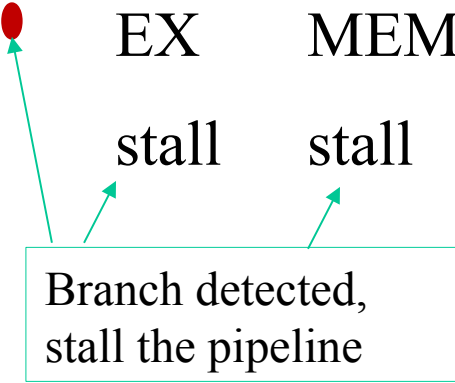
Control Hazards in Pipeline

- Control hazards are mainly caused by branch taken instructions.
- Control hazards cause a greater performance loss than data hazards do.
- An instruction is known as a branch after the ID stage.
- The simplest method is to stall the pipeline for preventing control hazards.

Instructions

clock cycles (cc)

	cc1	cc2	cc3	cc4	cc5	cc6	cc7
Branch	IF	ID	EX	MEM	WB		
Branch successor		IF	stall	stall	IF	ID	EX
Branch successor+1						IF	ID
Branch successor+2							IF



Branch detected,
stall the pipeline

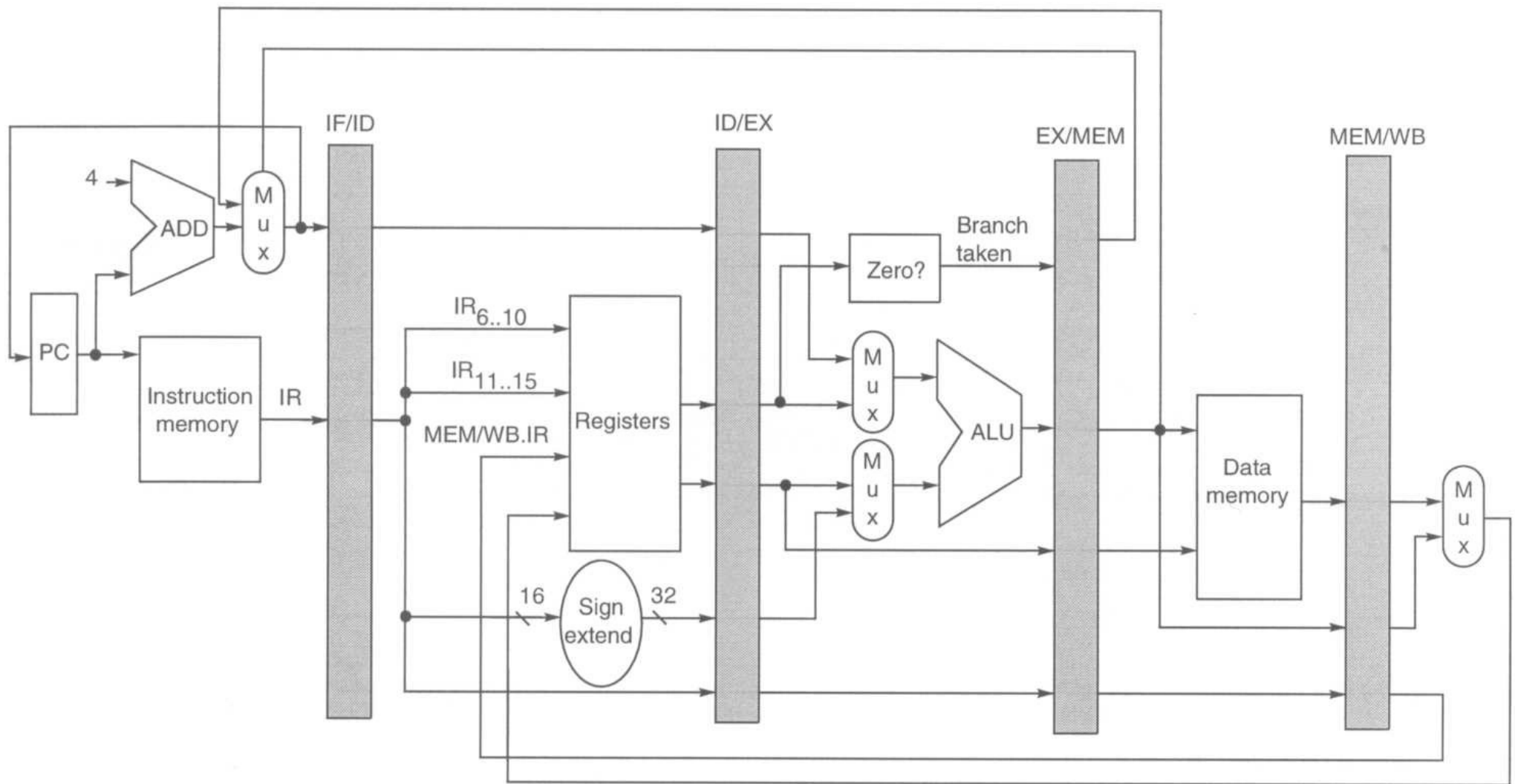
To reduce number of clock cycles in a branch stall:

- Find if branch is taken earlier in the pipeline
- Compute branch target address earlier to update PC

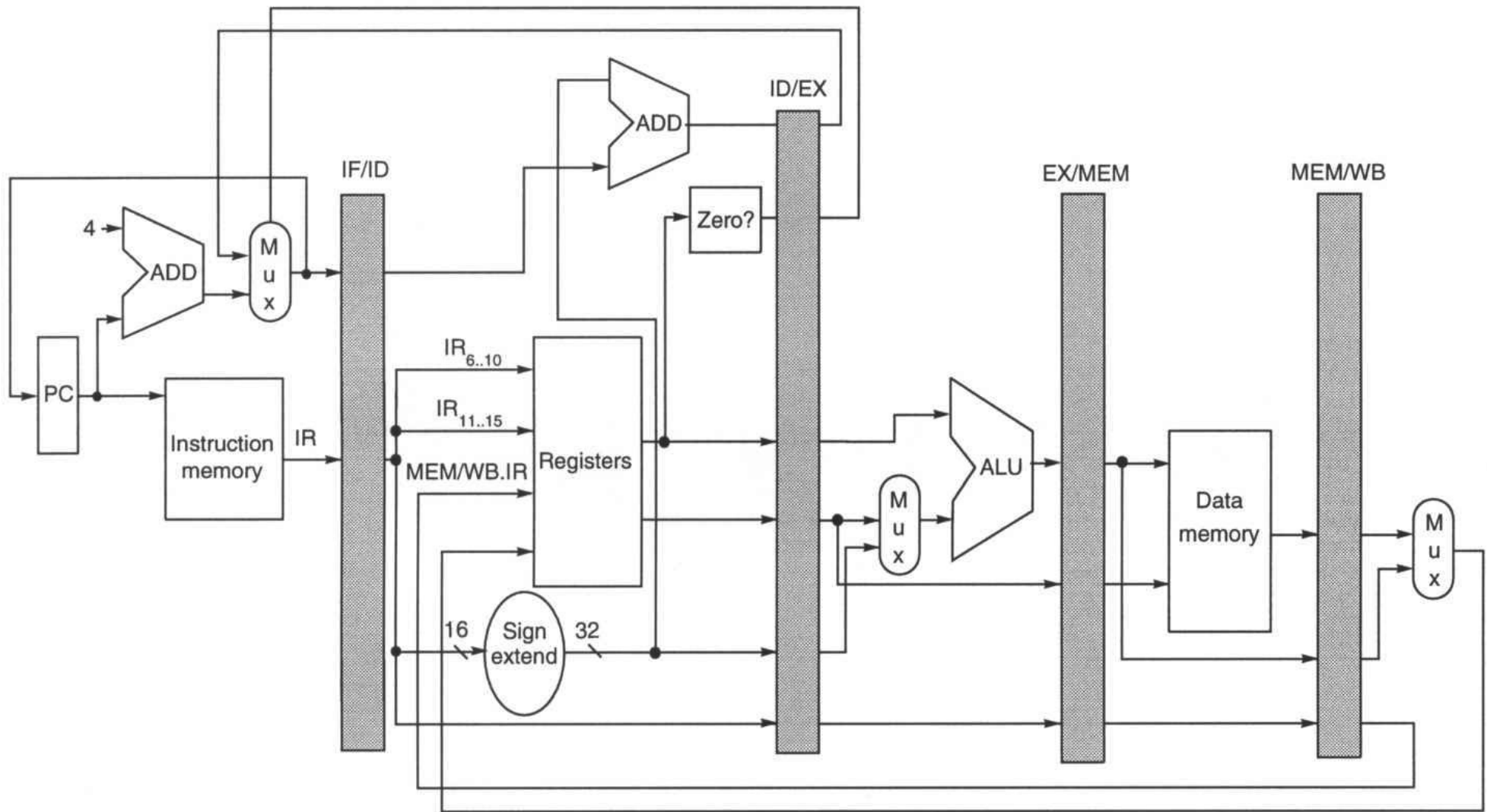
Control hazards in DLX

- Only 2 conditional branch instructions in DLX instructions: BEQZ and BNEZ;
- Move conditional testing into ID cycle;
- Update PC also at this stage, i.e. calculate branch target address in the ID cycle;
- This requires an additional adder in the ID cycle so that the pipeline datapath is modified.

Original 5-stage DLX pipeline datapath



Reducing stalls: the revised pipeline datapath



Static schemes for dealing with pipeline stalls caused by branch delays

- In these schemes, actions for a branch are static: they are fixed for each branch during the entire execution;
- We use the predict-untaken scheme in DLX;
- If it is taken, how to return to the instruction sequence?

With the revised pipeline, at the ID stage, the conditional test is done; and the branch target address calculated.

U.B.I	IF	ID	EX	MEM	WB					
I i+1		IF	ID	EX	MEM	WB				
I i+2			IF	ID	EX	MEM	WB			
I i+3				IF	ID	EX	MEM	WB		
I i+4					IF	ID	EX	MEM	WB	

T.B.I	IF	ID	EX	MEM	WB		
I i+1		IF	idle	idle	idle	idle	
Branch target			IF	ID	EX	MEM	WB
Branch target +1				IF	ID	EX	MEM
Branch target +2					IF	ID	EX
						MEM	WB

U.B.I: untaken branch instruction. It works as there is no branch;

T.B.I: taken branch instruction. If branch taken, stall the pipeline for one clock cycle

The delayed branch scheme

- The execution cycle with a branch delay of length n is

branch instruction

sequential successor 1

sequential successor 2

.....

sequential successor n

branch target if taken (instruction $i+2$ if not taken)

- The sequential successors are in the branch-delay slot (also called **branch delay instructions**), and executed whether or not the branch is taken.

cc	1	2	3	4	5	6	7	8	9
U.B.I	IF	ID	EX	MEM	WB				
B.D.I (i+1)	IF	ID	EX	MEM	WB				
I i+2			IF	ID	EX	MEM	WB		
I i+3				IF	ID	EX	MEM	WB	
I i+4					IF	ID	EX	MEM	WB

T.B.I	IF	ID	EX	MEM	WB				
B.D.I (i+1)	IF	ID	EX	MEM	WB				
Branch target			IF	ID	EX	MEM	WB		
Branch target +1				IF	ID	EX	MEM	WB	
Branch target +2					IF	ID	EX	MEM	WB

B.D.I: branch-delay instruction;

Note: The two examples work on the revised DLX pipeline datapath.

Dealing with control hazard: scheduling the branch-delay slot



Summary of the lecture

- We introduced structure hazards.
 - Resource conflict preventing an instruction executed in its designated cycle.
 - Use separate data and instruction memory ports
 - Stall the pipeline (insert bubbles).
- We also introduced control hazards.
 - Branch taken instructions cause control hazards
 - Revised pipeline datapath to reduce stall cycles
 - Static schemes to reduce time spent on stalling pipeline
 - Delayed branch scheme – branch delay instructions

End of this lecture

Questions and exercises

- What is pipeline structure hazard? List techniques used for dealing with it.
- What causes control hazards in a pipelined machine? List techniques used to reduce stall cycles in dealing with control hazards in a DLX pipelined machine.
- Explain the revised DLX pipeline data path in dealing with control hazards. What is the key purpose of revising the data path?